Step - 1: STM32F401RE Peripheral Driver Porting and Analysis

GPIO

1. Register-based Configuration

- **Clock Enable:** The `GPIO_PeriClockControl() - function enables or disables the peripheral clock for any given GPIO port (GPIOA to GPIOH) using RCC registers.
- **Mode Configuration:** In the `GPIO_Init()`- function, the `MODER` register configures the pin as input, output, alternate function, analog, or interrupt modes (Falling, Rising, or Both edges).
- **Speed:** The `OSPEEDR` register sets the output speed of the pin (Low, Medium, Fast, High).
- **Pull-up / Pull-down:** The `PUPDR` register controls the internal pull-up and pull-down resistors.
- **Output Type:** The `OTYPER` register sets the output type to either Push-Pull or Open-Drain.
- **Alternate Function:** The `AFR` (AFR[0], AFR[1]) registers configure the alternate functions ranging from AF0 to AF15.

2. Function Signatures & APIs

The driver header file provides the following major APIs:

- **Peripheral Clock setup:** GPIO_PeriClockControl()
- **Data read and write:** `GPIO_ReadFromInputPin()`, ‘GPIO_ReadFromInputPort()`,`GPIO_WriteToOutputPin()`, `GPIO_WriteToOutputPort()`, `GPIO_ToggleOutputPin()`
- **Init and De-init:** GPIO_Init(), GPIO_DeInit()
- **IRQ Configuration and ISR handling:** `GPIO_IRQInterruptConfig()`, `GPIO_IRQPriorityConfig()`, `GPIO_IRQHandling()`

3. Design Choices

- **Handle & Config Structures:** Hardware abstraction is cleanly implemented using `GPIO_PinConfig_t` to set pin numbers and modes, and `GPIO_Handle_t` to specify the GPIO port (GPIOA, GPIOB, etc.).
- **Interrupt Handling:** It uses a bare-metal approach by directly configuring EXTI registers (FTSR, RTSR, IMR, PR) and NVIC registers (ISER, ICER, IPR) for external interrupts.

I2C

1. Register-based Configuration

- **Clock Enable:** The I2C_PeriClockControl () function enables or disables the peripheral clock for I2C1, I2C2, or I2C3 using the RCC registers.
- **Initialization (I2C_Init)**
- **ACK Control**: Configures the acknowledgment bit via the CR1 register.
- **Peripheral Frequency**: Configures the FREQ field in the CR2 register based on the APB1 clock value to match timing requirements.
- **Own Address**: Sets the device's own address in the OAR1 register.
- **Clock Control (CCR)**: Calculates and sets the Clock Control Register (CCR) values for Standard Mode (Sm) or Fast Mode (Fm with Duty Cycle 2 or 16/9) based on the specified SCL speed.
- **TRISE Configuration:** Configures the maximum rise time register (TRISE) for both Standard and Fast modes.

2. Function Signatures & APIs

The driver header file provides the following primary APIs:

- **Peripheral Clock Setup:** I2C_PeriClockControl()
- **Initialization & De-initialization:** I2C_Init(), I2C_DeInit()
- **Data Transfer (Polling/Blocking):** I2C_MasterSendData(), I2C_MasterReceiveData()
- **Data Transfer (Interrupt-based):** I2C_MasterSendDataIT(), I2C_MasterReceiveDataIT(), I2C_SlaveSendData(), I2C_SlaveReceiveData()
- **Interrupt Configuration & ISR Handling:** I2C_IRQInterruptConfig(), I2C_IRQPriorityConfig(), I2C_EV_IRQHandling(), I2C_ER_IRQHandling()
- **Other Controls:** I2C_PeripheralControl(), I2C_GetFlagStatus(), I2C_ManageAcking(), I2C_GenerateStopCondition()

3. Design Choices & Implementation Details

- **Hardware Abstraction:** Utilizes configuration (I2C_Config_t) and handle (I2C_Handle_t) structures to separate user configuration parameters from runtime state variables (such as Tx/Rx buffer pointers, lengths, and communication states).
- **Polling (Blocking) Mode:** Functions like I2C_MasterSendData use while loops to monitor status flags (SB, ADDR, TXE, RXNE, BTF) causing the processor to wait until transmission or reception finishes.
- **Interrupt-Based Mode:** Functions like I2C_MasterSendDataIT enable event, buffer, and error interrupts (ITBUFEN, ITEVFEN, ITERREN) in the CR2 register, allowing non-blocking asynchronous data communication handled via I2C_EV_IRQHandling() and I2C_ER_IRQHandling().

- **Error Handling:** The driver explicitly handles and clears I2C bus errors (BERR), arbitration lost errors (ARLO), acknowledge failures (AF), overrun/underrun errors (OVR), and timeouts (TIMEOUT).

SPI

1. Register-based Configuration

- **Clock Enable:** The SPI_PeriClockControl() function enables or disables the peripheral clock for SPI1, SPI2, SPI3, or SPI4 using the RCC registers.
- **Initialization (SPI_Init)**
- **Device Mode:** Configures the master or slave configuration using the MSTR bit in the CR1 register.
- **Bus Configuration:** Sets up Full-Duplex, Half-Duplex (by configuring the BIDIMODE bit), or Simplex RX-Only modes (by configuring BIDIMODE and RXONLY bits).
- **Clock Speed (Baud Rate):** Configures the serial clock speed using the BR[2:0] bits in the CR1 register.
- **Data Frame Format (DFF):** Configures 8-bit or 16-bit data formats via the DFF bit.
- **Clock Polarity and Phase:** Configures CPOL and CPHA bits to define the clock behavior.
- **Slave Management:** Configures Software Slave Management (SSM).

2. Function Signatures & APIs

The driver header file provides the following primary APIs:

- **Peripheral Clock Setup:** SPI_PeriClockControl()
- **Initialization & De-initialization:** SPI_Init(), SPI_DeInit()
- **Data Transfer (Polling/Blocking):** SPI_SendData(), SPI_ReceiveData()
- **Data Transfer (Interrupt-based):** SPI_SendDataIT(), SPI_ReceiveDataIT()
- **Interrupt Configuration & ISR Handling:** SPI_IRQInterruptConfig(), SPI_IRQPriorityConfig(), SPI_IRQHandling()
- **Other Controls:** SPI_PeripheralControl(), SPI_SSIConfig(), SPI_SSOEConfig(), SPI_GetFlagStatus(), SPI_ClearOVRFlag()

3. Design Choices & Implementation Details

- **Hardware Abstraction**: Implements configuration (SPI_Config_t) and handle (SPI_Handle_t) structures to cleanly separate static configuration parameters from dynamic runtime variables (like buffer pointers, lengths, and transmission/reception states).
- **Blocking vs. Interrupt-Based Implementation:**
- **Polling (Blocking) Mode:** Functions like SPI_SendData and SPI_ReceiveData use while loops to poll the TXE and RXNE flags in the status register (SR), blocking execution until data transfer completes. It also handles 8-bit and 16-bit data frame formats dynamically by casting buffer pointers.

- **Interrupt-Based Mode:** Functions like SPI_SendDataIT and SPI_ReceiveDataIT enable respective interrupt control bits (TXEIE, RXNEIE, and ERRIE) in the CR2 register for asynchronous, non-blocking communication handled inside SPI_IRQHandling() using helper routines.
- **Error Handling:** Features built-in handling for Overrun (OVR) error conditions via spi_ovr_err_interrupt_handle() and flags clearing to ensure communication stability.

USART

1. Register-based Configuration

- **Clock Enable:** The USART_PeriClockControl() function enables or disables the peripheral clock for USART1, USART2, USART3, UART4, UART5, or USART6 using the appropriate APB bus clock registers.
- **Baud Rate Configuration (USART_SetBaudRate):** Automatically computes and sets the mantissa and fraction components in the baud rate register (BRR) based on the peripheral bus clock (PCLK1 or PCLK2) and oversampling settings (OVER8 = 0 or 1).
- **Initialization (USART_Init):**
- **CR1 Register:** Configures the word length (8 or 9 bits), parity control (Even, Odd, or Disabled), and enables the transmitter (TE) and receiver (RE) engines according to the USART_Mode configuration.
- **CR2 Register:** Sets the number of stop bits (USART_NoOfStopBits).
- **CR3 Register:** Configures hardware flow control options (CTS, RTS, or both).

2. Function Signatures & APIs

The driver header file provides the following primary APIs:

- **Peripheral Clock Setup:** USART_PeriClockControl()
- **Initialization & De-initialization:** USART_Init(), USART_DeInit()
- **Data Transfer (Polling/Blocking):** USART_SendData(), USART_ReceiveData()
- **Data Transfer (Interrupt-based):** USART_SendDataIT(), USART_ReceiveDataIT()
- **Interrupt Configuration & ISR Handling:** USART_IRQInterruptConfig(), USART_IRQPriorityConfig(), USART_IRQHandling()
- **Other Controls:** USART_GetFlagStatus(), USART_ClearFlag(), USART_PeripheralControl(), USART_SetBaudRate()

3. Design Choices & Implementation Details

- **Hardware Abstraction:** Utilizes configuration (USART_Config_t) and handle (USART_Handle_t) structures to cleanly separate application configurations from runtime status variables (such as buffer pointers, lengths, and busy flags).
- **Blocking vs. Interrupt-Based Implementation**

- **Polling (Blocking) Mode:** Functions like USART_SendData and USART_ReceiveData loop using while statements to monitor the TXE, RXNE, and TC flags in the status register (SR), halting execution until the entire buffer is transferred.
- **Interrupt-Based Mode:** Functions like USART_SendDataIT and USART_ReceiveDataIT configure data pointers, update busy states, and enable transmission/reception interrupt enable flags (TXEIE, TCIE, RXNEIE) in CR1 for asynchronous, non-blocking communication managed inside USART_IRQHandling().
- **Error and Event Handling:** The ISR handles transmission complete (TC), data register empty (TXE), read data register not empty (RXNE), clear-to-send (CTS), idle line detection (IDLE), overrun error (ORE), noise error (NE), and framing error (FE) flags, invoking application callbacks accordingly.